



BACHELORARBEIT

DEVELOPMENT OF A MULTI-AGENT PIPELINE UTILIZING LARGE LANGUAGE MODELS FOR COMPLEX TASK AUTOMATION

Verfasser

Sergei Ovsianikov

angestrebter akademischer Grad

Bachelor of Science (BSc)

Wien, 2026

Studienkennzahl lt. Studienblatt: A 033 521

Fachrichtung:

Informatik - Data Science

Betreuerin / Betreuer:

Dipl.-Ing. Dr.techn. Marian Lux

Contents

1	Motivation	4
1.1	Problem statement	4
1.2	Goals of the Work	4
1.3	Process and Methods	4
2	Related Work	5
2.1	Large Language Models (LLMs)	5
2.2	Limitations of LLMs	6
2.3	AI Agents	6
2.4	Multi-Agent Frameworks	7
2.5	Architectural Patterns of Agents Orchestration	8
3	Design	8
3.1	Architecture of the Framework Core	8
3.2	Architecture of the Use Case Trip Planning	9
3.3	Alternatives	12
4	Implementation	12
4.1	Structure of Repository	12
4.2	<code>core</code> Directory	13
4.3	<code>adapters</code> Directory	15
4.4	<code>examples</code> Directory	15
4.5	<code>tools</code> Directory	17
4.6	Other Files	18
5	How to Use	18
5.1	Requirements	18
5.2	Setup Ollama	19
5.3	Install Package	19
5.4	Verify Installation	19
5.5	Run Pipelines	20
5.5.1	Option 1: With Defined Model and Logger (Recommended)	20
5.5.2	Option 2: Without Logger	20
5.5.3	Available tools configuration	20
6	How to Maintain	21
6.1	Creating a Function Node	21
6.2	Creating an Agent Node	21
6.3	Building a Pipeline	21
6.3.1	Merging Pipelines	22
6.3.2	Nested Pipelines	22
6.3.3	Edge Conditions	23
6.3.4	Creating Loops	23
6.3.5	Creating Adapters	24

6.4	Run Test Scripts	24
6.4.1	Option 1: Run a Single Pipeline	24
6.4.2	Option 2: Run All Example Pipelines (Smoke Tests)	24
7	Evaluation	25
7.1	Evaluation Approach	25
7.2	Preparation	25
7.3	Evaluation Results	26
7.3.1	Qwen3 Model with All Tools	26
7.3.2	Llama3.2 Model with All Tools	27
7.3.3	Llama3.2 Model without Tools	27
7.3.4	Llama3.2 Model with Tools for Local Expert only	28
8	Conclusion and Future Work	28
8.1	Limitations	29
8.2	Future Work	29

Abstract

This thesis presents the design and implementation of a modular, pipeline-oriented framework that coordinates AI agents through explicit data flows and deterministic control logic. The suggested method enhances transparency and permits effective execution on lightweight and locally hosted Large Language Models (LLMs), in contrast to many existing multi-agent frameworks that rely on implicit agent autonomy and heavyweight architectures. The proposed framework uses an adapter-based architecture and a unified node abstraction to combine deterministic function execution with LLM-based reasoning. Several example pipelines, including a trip planning use case, are used to implement and assess a prototype. According to the experimental results, the framework works well with smaller local LLMs, providing developers with strong control over workflow and context while still being transparent and maintainable.

1 Motivation

1.1 Problem statement

Large language models’ quick development has made it possible to automate ever-more-complex tasks, especially when using AI agents that can plan, reason, and communicate with outside tools. The flexibility and maintainability of current agent-based systems are, however, frequently constrained by tightly coupled architectures, heavyweight frameworks, or strong assumptions about model capabilities. As a result, there is a disconnect between theoretical agent-based automation and realistic, resource-efficient implementations that can be used with locally hosted or lightweight LLMs. Therefore, a more controllable and modular method of coordinating AI agents in intricate workflows is required.

1.2 Goals of the Work

The primary goal of this thesis is to create and put into use a lightweight, easily maintainable framework that can be adjusted to various execution environments for creating and running AI-agent-based pipelines. By supporting small and locally hosted LLMs, the framework seeks to lower external dependencies and computational demands. Modularity, distinct concern separation, and explicit data flow between processing steps are emphasized. Furthermore, the framework is designed to be extensible, making it easy to incorporate new agents, tools, and control logic.

1.3 Process and Methods

The work starts by looking at current multi-agent frameworks, paying special attention to their strengths, weaknesses, and architectural patterns. From this analysis, a custom pipeline-based architecture is created that allows for deterministic execution, conditional branching, and agent integration through

adapters. We built and tested a working prototype of the framework using a few example pipelines, one of which was a trip-planning scenario. Finally, the behavior, performance, and maintainability of the proposed solution are experimentally compared to CrewAI, highlighting trade-offs between flexibility, transparency, and execution overhead.

2 Related Work

This project employs local Large Language Models together with web search and content-retrieval tools to demonstrate their capabilities in a concrete example of a proposed architecture. This chapter covers the basic concepts and terminology relevant to this project.

2.1 Large Language Models (LLMs)

As described by Andrej Karpathy, a large language model can be interpreted as a token-level simulation of internet documents¹. LLMs can be characterized as neural networks trained on vast amounts of textual data to predict the next token in a sequence.[1] Through this probabilistic next-token prediction, LLMs learn linguistic structures, semantic relationships, and contextual patterns, enabling them to generate coherent and context-aware natural language outputs.

In most contemporary applications, large language models (LLMs) are accessed via an Application Programming Interface (API), where user requests are sent to servers operated by model providers such as OpenAI (ChatGPT), Google (Gemini), or DeepSeek. While this approach is convenient, it is not always optimal. In many use cases, deploying LLMs locally is preferable, particularly when working with confidential or sensitive data that should not leave the local infrastructure. Additionally, from a production perspective, running local LLMs can be significantly more cost-efficient than relying on cloud-based APIs, especially for frequent or large-scale inference workloads.²

As the default model for this project, a locally deployed Llama model managed via *Ollama* was selected. Ollama serves as a lightweight local LLM aggregator that enables easy model installation and execution without requiring high-end hardware. In particular, the selected setup runs comfortably on a standard machine with approximately 8 GB of RAM, making it suitable for resource-constrained environments.³ While there are numerous commercial providers - most notably OpenAI, whose models are widely adopted - they typically do not allow local model deployment and often require paid subscriptions, which limits their suitability for the goals of this project.⁴

¹ Andrej Karpathy: *Deep Dive into LLMs like ChatGPT (Youtube)*: <https://www.youtube.com/watch?v=7xTGNLPyMI>

² Neil Sahota: <https://www.neilsahota.com/local-llm-when-running-ai-in-house-becomes-the-smarter-choice/>

³ Ollama: <https://ollama.com/>

⁴ OpenAI Pricing: <https://openai.com/pricing>

Another alternative is Google Gemini; however, it was not chosen due to limited support for local deployment and a strong dependency on proprietary cloud infrastructure.⁵ At the time of writing, the most recent generation of Llama models is Llama 4. However, this version is currently unavailable for use within the European Union due to regulatory restrictions related to its multimodal capabilities.⁶

Earlier versions, such as Llama 3.3, are publicly available but require substantial storage and computational resources, with model sizes exceeding 40 GB, making them impractical for lightweight local execution.⁷ To balance model capability and resource efficiency, Llama 3.2 was selected as the default model for this framework. Llama 3.2 offers a variant with model size 2 GB, making it a good fit for running on typical consumer machines, even in the absence of GPUs.⁸

Specifically, the `llama3.2:latest` variant with approximately three billion parameters and a weight size of around 2 GB is used by default. Nevertheless, the framework itself is model-agnostic and this default configuration can easily be replaced with any other Ollama-supported model if required by a particular use case.

2.2 Limitations of LLMs

However, LLMs have access only to information available up to the end of their training period. Consequently, they lack knowledge of real-world events occurring after this cutoff, such as recent news developments, future weather conditions, or the outcomes of sporting events that took place after training completion. They are also constrained by the number of hyperparameters. A larger number of these parameters makes a model more expressive but also more computationally expensive, requiring stronger graphics processing units (GPUs) to run, and vice versa.

Moreover, recent research has shown that LLMs achieve the best performance when they are concentrated on a concrete task, have a concrete role, and have a relevant context [2]. As benchmarks indicate[3], orchestration is at least as critical as the LLM’s raw intelligence, if not even more so.

2.3 AI Agents

This is where AI agents become relevant. Rather than relying on a single, general-purpose LLM, a complex task can be decomposed into multiple smaller sub-tasks, each handled by a specialized agent equipped with appropriate context and tools. By orchestrating such agents, each focused on a well-defined sub-problem, complex objectives can be addressed more effectively, resulting in improved robustness, modularity, and overall automation quality [4].

⁵ *Google Gemini*: <https://deepmind.google/technologies/gemini/>

⁶ *Zansara*: <https://www.zansara.dev/posts/2025-05-16-llama-eu-ban/>

⁷ *Ollama Llama 3.3*: <https://ollama.com/library/llama3.3>

⁸ *Ollama Llama 3.2*: <https://ollama.com/library/llama3.2>

AI agents extend beyond pure language generation by enabling goal-directed interaction with external tools [5]. Typical tools include web scrapers for extracting structured information from websites and web search APIs for accessing up-to-date or long-tail content. In the implemented system, web search functionality is realized using the DDGS python package⁹ (Dux Distributed Global Search) based on DuckDuckGo Search engine¹⁰, which was selected due to its broad market adoption, the absence of an API key requirement, and the lack of usage restrictions, in contrast to alternative providers such as Serper¹¹.

For web scraping, the BeautifulSoup library¹² was employed. BeautifulSoup offers a lightweight and flexible approach to HTML parsing, with low overhead, strong fault tolerance for malformed markup, and seamless integration with Python-based pipelines, making it well suited for rapid prototyping compared to heavier frameworks such as Scrapy¹³ or browser-based automation tools.

In addition, Retrieval-Augmented Generation (RAG) components can allow agents to retrieve relevant documents from private or public knowledge bases. However, RAG is not currently integrated into the system, as it was not required for the given use case. The architecture nevertheless allows for RAG to be incorporated in the future if the need for contextual document retrieval arises.

Through these capabilities, agents can observe their environment (e.g., query, search, or scrape), reason over the acquired information, and act by invoking APIs, triggering workflows, or updating data stores, while maintaining intermediate state and dynamically adapting their strategy.

2.4 Multi-Agent Frameworks

Several multi-agent frameworks have emerged that support the definition, coordination, and execution of multiple interacting AI agents through structured workflows. These frameworks decompose complex tasks into specialized agent roles and manage their communication, internal state, and control flow, enabling planning, tool usage, and multi-step reasoning.

LangGraph¹⁴ was introduced around mid-2023 as an extension of the ecosystem LangChain¹⁵ and quickly became a prominent framework for agent orchestration. It is an open-source, low-level agent orchestration framework designed for building controllable multi-agent systems, emphasizing structured workflows and scalability. A major alternative is CrewAI¹⁶, which focuses on collaborative multi-agent systems and workflow automation, enabling coordinated task execution among agents and supporting practical enterprise and research use cases. There are other alternatives: OpenAI Swarm¹⁷ adopts a collaborative AI

⁹ DDGS python package: <https://pypi.org/project/ddgs/>

¹⁰ DuckDuckGo Search: <https://duckduckgo.com/>

¹¹ Serper: <https://serper.dev>

¹² BeautifulSoup: <http://crummy.com/software/BeautifulSoup/bs4/doc/>

¹³ Scrapy framework: <https://www.scrapy.org/>

¹⁴ LangGraph framework: <https://github.com/langchain-ai/langgraph>

¹⁵ LangChain ecosystem: <https://www.langchain.com/>

¹⁶ CrewAI framework: <https://github.com/crewAIInc/crewAI>

¹⁷ OpenAI Swarm framework: <https://github.com/openai/swarm>

paradigm where multiple agents interact to solve complex problems, achieving strong performance in latency and throughput for real-time applications. For integration with cloud infrastructure, Google ADK¹⁸ (discussed in the context of emerging enterprise and cloud-integrated agent platforms) aligns with large-scale deployment and cloud ecosystem integration, targeting enterprise-grade automation and scalable AI agent orchestration.[6]

2.5 Architectural Patterns of Agents Orchestration

Agent orchestration patterns can be grouped into role-based team orchestration, graph/state-machine orchestration, conversational or emergent coordination, and declarative/service-oriented orchestration.[7] CrewAI exemplifies role-based orchestration with explicit agent roles and delegation; LangGraph represents graph-based, stateful orchestration; OpenAI Swarm / Agents SDK and AutoGen align with conversational and emergent multi-agent coordination; while Semantic Kernel and Google ADK fit declarative, enterprise-oriented orchestration focused on planners, skills, and service integration.

The project’s architecture relies on graph-based orchestration, enabling fine-grained control over dataflow and thus allowing the data to be tailored for lightweight LLMs. At the same time, the nodes in this graph can host agents or even pipelines from other frameworks (which do not need to be graph-based), and they integrate with CrewAI out of the box. The next chapter will discuss the architecture in more detail.

3 Design

This chapter presents the architecture of the core framework and the target use case it is designed to support. The framework is structured to simplify the construction of modular and reusable processing pipelines while remaining compatible with CrewAI and extensible to other external agent frameworks. It is designed to operate fully locally with lightweight LLMs, using Llama 3.2 as the default model. Finally, the chapter demonstrates how this architecture is applied to a real-world use case to validate its practical applicability.

3.1 Architecture of the Framework Core

The proposed framework is based on a graph-based pipeline architecture (Figure 1) in which each pipeline consists of a sequence of *Nodes* that process data step by step. Each node takes a defined set of input values from the shared context and produces exactly one output value, which is then added back to the context. The central design goal is to enable the construction of maintainable and transparent processing schemas, where the data flow between nodes is explicit and easy to trace. In the current implementation, the framework provides two concrete node types: *FunctionNode* and *AgentNode*. A *FunctionNode*

¹⁸Google ADK framework: <https://github.com/google/adk-python>

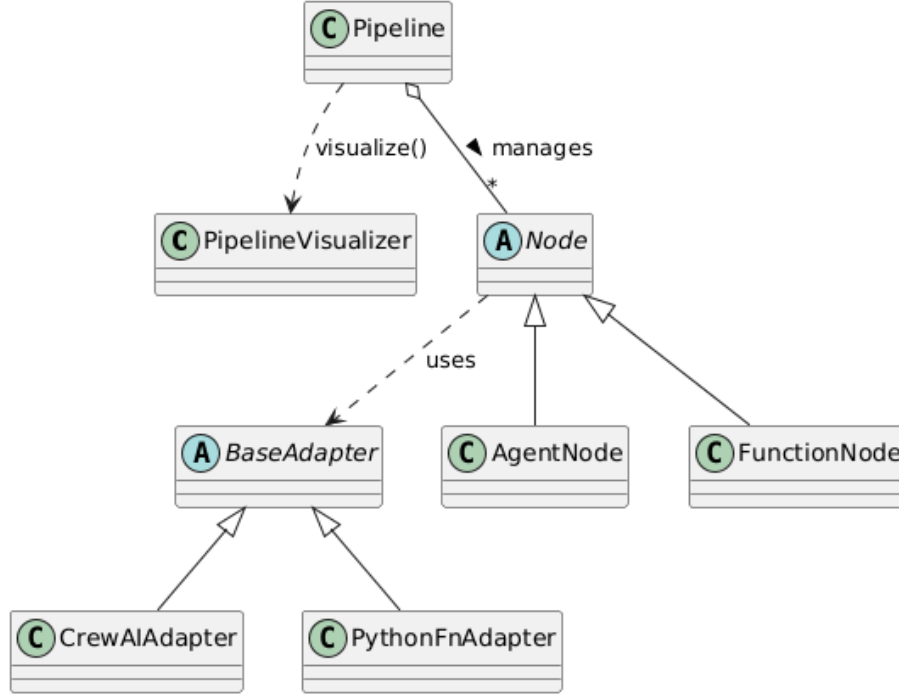


Figure 1: Class diagram of the framework core (only class names and relations)

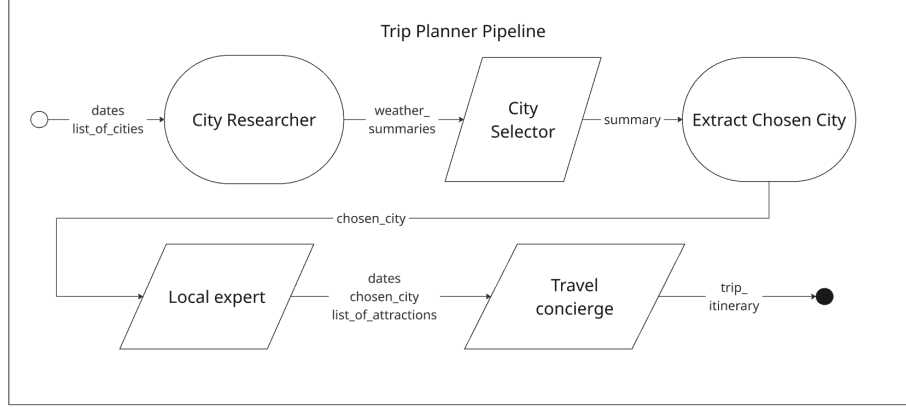
executes a deterministic Python function via a corresponding adapter, while an **AgentNode** delegates processing to an LLM through an adapter abstraction. The **Pipeline** class is responsible for managing the node execution order and can optionally visualize the pipeline structure using a dedicated **PipelineVisualizer** component.

This architecture is designed to be extensible. New node types can be introduced by inheriting from the abstract base class **Node**, and additional execution backends can be integrated by creating new adapters that inherit from **BaseAdapter** according to *adapter pattern*¹⁹. This separation of concerns between orchestration (nodes and pipelines) and execution logic (adapters) allows the framework to support a wide range of use cases, from purely deterministic data processing to agent-based reasoning with LLMs.

3.2 Architecture of the Use Case Trip Planning

One representative use case of the proposed framework, suitable for execution with lightweight local LLM, is a trip planning scenario in which contextual

¹⁹ *Adapter pattern on Refactoring Guru*: <https://refactoring.guru/design-patterns/adapter>



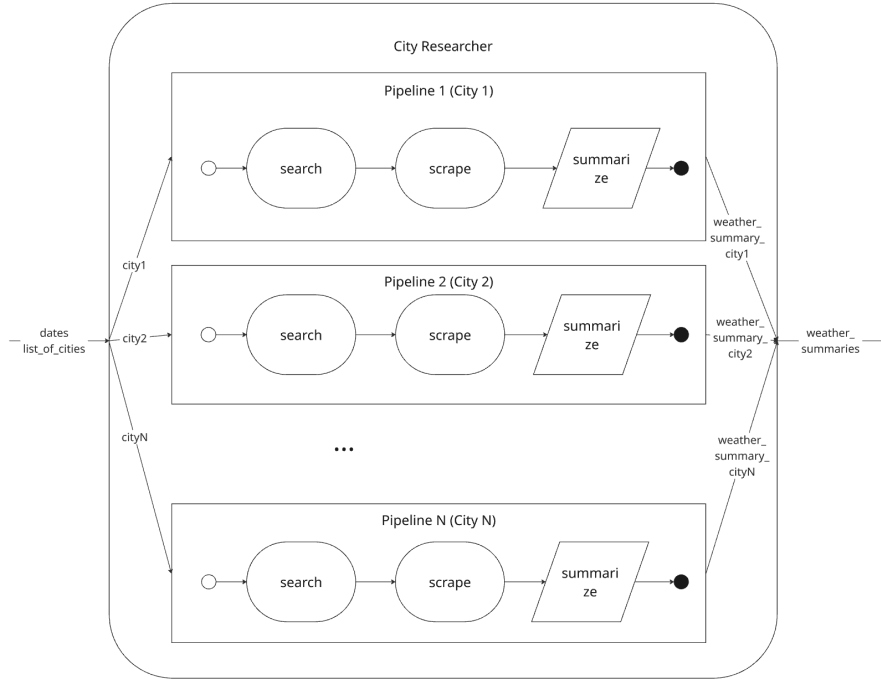
Legend:

- Rounded rectangle – function node
- Parallelogram – agent node
- Rectangle – pipeline
- White circle – beginning of pipeline
- Filled circle – end of pipeline
- Arrow – data flow

Figure 2: Trip planning pipeline diagram with execution flow.

information is partially retrieved from the internet. The general execution flow of this use case is illustrated in Figure 2. The pipeline starts with user input consisting of a list of candidate cities and a desired travel time span. Based on this input, the system first retrieves weather information for all specified cities within the given dates using the *City Researcher* node. Next, the *City Selector* agent evaluates the collected information and selects the city with the most favorable weather conditions. The selected city is then explicitly extracted from the agent output to enable further deterministic processing. Subsequently, the *Local Expert* node identifies points of interest in the chosen city, and finally the *Travel Concierge* agent generates a detailed seven-day travel itinerary.

Since the *City Researcher* node repeatedly performs the same type of information retrieval for different cities, this step is modeled as a loop over a nested pipeline, where each iteration processes exactly one city. The internal structure of this nested pipeline is shown in Figure 3. For each city, a web search is performed using a dedicated function node, followed by content extraction from the retrieved web pages via a scraper node. To ensure that only relevant and compact information is passed to subsequent decision-making stages, the collected data is summarized using an agent node before being returned to the outer pipeline. This design allows the framework to combine deterministic data collection with LLM-based reasoning while keeping the overall execution transparent and efficient.



Legend:

- Rounded rectangle – function node
- Parallelogram – agent node
- Rectangle – pipeline
- White circle – beginning of pipeline
- Filled circle – end of pipeline
- Arrow – data flow

Figure 3: Diagram of City Researcher node with execution flow.

3.3 Alternatives

Several alternative design approaches were considered during development. One option was to allow nodes to produce multiple output values and modify several fields of the shared context simultaneously. However, this approach quickly led to ambiguities in data dependencies and made context tracing difficult, especially in larger pipelines. To avoid this complexity, the final design enforces a single-output rule for each node, which significantly improves transparency and debuggability.

Another alternative was the use of a static, globally shared context that nodes would modify in place without returning explicit outputs. This approach was rejected because it violated the open-closed principle and tightly coupled node implementations to a specific context structure. Instead, the framework adopts a dynamic context dictionary that is updated incrementally by each node. As a result, different pipelines can define and evolve their own context schemas independently, improving modularity, reusability, and long-term maintainability.

4 Implementation

In this chapter, we discuss the implementation details. The full source code for this project is publicly available on GitHub.²⁰

4.1 Structure of Repository

```
pipeline_node_agents/  
├── src/pipeline_node_agents/  
│   ├── core/  
│   │   ├── node.py          # FunctionNode, AgentNode definitions  
│   │   └── pipeline.py      # Pipeline orchestration  
│   ├── adapters/           # Adapters for different execution backends  
│   └── examples/           # Example pipelines  
├── docs/                   # Documentation  
├── scripts/                # Bash scripts for test execution  
├── logs/                   # Logs, saved locally  
├── pyproject.toml  
└── README.md
```

Figure 4: Structure of project repository

The project follows the standard `src` layout required for PyPI packaging, with the main package code located in `src/pipeline_node_agents/`—the duplicated name ensures that the package is properly importable after installation via `pip install pipeline_node_agents`. Inside this package, `core/` contains

²⁰ Project source code: https://github.com/ser0vs/pipeline_node_agents

the fundamental node and pipeline classes, `adapters/` provides pluggable integrations for various AI backends, and `examples/` offers ready-to-use pipeline templates. The root-level `pyproject.toml` defines all package metadata, dependencies, and build configuration required for publishing to PyPI, while supporting directories such as `docs/`, `scripts/`, and `logs/` remain outside the package and are therefore not included in the distributed release.

4.2 core Directory

node.py The file `node.py` defines the fundamental building blocks of every pipeline: `FunctionNode` for executing deterministic Python functions and `AgentNode` for running AI-powered tasks through adapters. Both node types follow the same execution pattern: they extract required inputs from a shared context dictionary, invoke their adapter logic, and update the context with their *single* output. If the parameter `mock_value` is provided, the node returns this mock value instead of executing the adapter logic. The abstract base class `Node` (Figure 5) enforces a consistent interface by requiring all nodes to implement a `run(context)` method that both receives and returns the pipeline’s shared state.

```
class Node(ABC):
    """Abstract base class for all nodes in the pipeline."""
    def __init__(self, name, adapter=None, inputs=None, output=None, mock_value=None):
        self.name = name
        self.adapter = adapter
        self.inputs = inputs or []
        self.output = output
        self.mock_value = mock_value

    @abstractmethod
    def run(self, context: dict) -> dict:
        """Execute the node logic using given context."""
        pass
```

Figure 5: Code Snippet of base class `Node` from `node.py`

pipeline.py The file `pipeline.py` orchestrates node execution and manages the flow of data between nodes via a shared context dictionary. It supports two construction modes: a simple linear pipeline created from a list of nodes (with sequential edges generated automatically), and a flexible graph-based pipeline with explicit edges and optional conditions to enable branching logic. During execution, the pipeline starts from a designated entry node and follows outgoing edges based on condition evaluation, continuing until no further edges match. This approach enables complex workflows with conditional branching and iterative loops.

Figure 6 describes the functions of class `Pipeline` for building pipeline. For graph-based pipelines, the structure can be built incrementally using `add_node()`

```

def add_node(self, node):
    self.nodes[node.name] = node

def add_edge(self, from_node, to_node, condition=None):
    """
    condition: None | callable(context) -> bool
    """
    self.edges.setdefault(from_node, []).append((condition, to_node))

def add_pipeline(self, other_pipeline):
    """
    Adds all nodes and edges from another pipeline into this pipeline.
    """

    for node_name, node in other_pipeline.nodes.items():
        if node_name in self.nodes:
            raise ValueError(f"Node '{node_name}' already exists in pipeline")
        self.nodes[node_name] = node

    for from_node, transitions in other_pipeline.edges.items():
        for condition, to_node in transitions:
            self.edges.setdefault(from_node, []).append((condition, to_node))

```

Figure 6: Code Snippet of methods for pipeline creation of class Pipeline from pipeline.py

to register individual nodes and `add_edge()` to define transitions between them. Each edge may optionally include a condition callable that determines whether the corresponding path should be taken based on the current context. The `add_pipeline()` method enables pipeline composition by merging all nodes and edges from another pipeline instance. This allows reusable sub-pipelines to be combined into larger workflows while explicitly checking for naming conflicts. During execution, the pipeline starts from a designated entry node and follows outgoing edges according to condition evaluation, continuing until no further edges match. This design enables complex workflows featuring conditional branching and iterative loops.

pipeline_visualizer.py The file `pipeline_visualizer.py` provides a CLI-based visualization of pipeline graphs to aid developers in understanding execution flow. The algorithm traverses the graph starting from the initial node, following the primary (first) edge at each step to construct the *main path*, while collecting side branches and detecting loops when a node is revisited. The resulting visualization renders the main path as a horizontal arrow chain and prints side branches using indentation, marking detected loops with the circled arrow symbol. Figure 7 shows an example of `pipeline_visualizer` output with main branch of Nodes "Start", "A", "B", "C" and secondary branch from "A" to "X" and loop from "X" to "A".

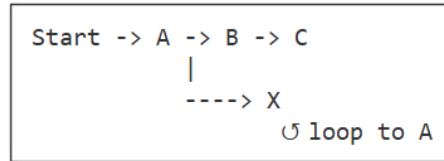


Figure 7: Example of `pipeline_visualizer` output with loop and branching

other files The file `logger_bootstrap.py` initializes pipeline-specific logging by creating timestamped log files within a structured directory under the project root. The file `logging_config.py` configures Python’s logging system with a consistent format and supports both console and file output handlers. Finally, `paths.py` provides a utility function for dynamically resolving the project root by traversing the directory hierarchy upward until either `pyproject.toml` or `.git` is located.

4.3 adapters Directory

The `adapters/` directory implements the Adapter pattern to decouple pipeline nodes from specific execution backends. The abstract base class `BaseAdapter` defines a uniform interface with a single `invoke(**kwargs)` method that all adapters must implement, ensuring that nodes can interact with any backend in a consistent manner.

The `PythonFnAdapter` wraps plain Python functions for use within `FunctionNode`, forwarding keyword arguments directly to the wrapped function and returning its result.

The `CrewAIAdapter` enables integration with the CrewAI framework. It is designed to coordinate CrewAI agents as agent-nodes within the graph. `CrewAIAdapter` serves as the default adapter for all AI agents and can handle both `Agent` instances and `Crew` instances (collections of Agents managed by CrewAI). All agent-specific logic is supplied by CrewAI through the `CrewAIAdapter`.

4.4 examples Directory

The `examples` folder contains ready-to-use pipeline implementations that demonstrate various architectural patterns and serve as templates for building custom workflows. As provided in Project’s README:

The following pipelines are included as examples:

- `ConditioningPipeline`: Randomly chooses whether to go to a park or cinema, and suggests either a film or a park in Vienna.
- `InputCheckerPipeline`: Takes text as input and returns true if the text is a list of populated places (cities). Can be run as a loop, returning to the input step as long as the input is invalid: `pipeline.run(loop=True)`

- **RandomMeanPipeline:** Simple pipeline without LLM usage that generates random numbers and calculates their mean.
- **RandomMeanPipelineCrewAI:** Generates random numbers and summarizes them using an LLM.
- **SearchAndSummarizePipeline:** Searches for the best country for business using DuckDuckGo and summarizes results using an LLM, returning the best country for business.
- **TripPlannerPipeline:** Takes a list of cities and trip dates as input, chooses the city based on weather conditions, and creates a full 7-day trip itinerary for the provided dates.

Source: Project README file, https://github.com/ser0vs/pipeline_node_agents/blob/master/README.md

In these examples, the ‘Agent’ class from CrewAI is employed as the underlying engine for agents. Some examples showcase tools that Agents can leverage, whereas most rely on the ‘Agent’ class as a plain LLM to make examples not too complex. Further details on how agents use tools are provided in the Chapter 7 – Evaluation.

Each pipeline class is implemented in a Python file that uses the same name in lower case, with the sole exception of `TripPlannerPipeline`. Let us examine this one in more detail.

The `trip_planner/` directory contains the files required to run `TripPlannerPipeline`. The file `config.py` contains the shared LLM configuration, whereas `agents.py` defines CrewAI agents responsible for summarization, city selection, local expertise, and travel planning providing them with tools “websearch”, “scrape” and “calculator”.

Unlike the standard CrewAI approach, these agents are not grouped into a single Crew; instead, they are separated into individual agent nodes within a graph-based orchestration used in this project. Chapter 7 compares this graph-based orchestration – where some agents are augmented with function nodes – with the conventional CrewAI orchestration based on a Crew.

The file `functions.py` contains not only helper functions such as city extraction and formatting, but also nested pipeline which runs in a loop in the function `research_cities`, and `nodes.py` wraps these agents and functions into reusable `FunctionNode` and `AgentNode` instances.

Finally, `pipeline.py` orchestrates all components into the main `TripPlannerPipeline` class.

To execute the `TripPlannerPipeline`, as provided on Figure 8, three high-level components are required:

- **Pipeline with Nodes:** A `Pipeline` object is created using an ordered sequence of nodes from `TripPlannerNodes`. These include research, city selection, extraction, local expert recommendation, and travel concierge planning nodes, which are executed sequentially.


```

main_pipeline = Pipeline(nodes=[
    TripPlannerNodes.get_research_cities_node(),
    TripPlannerNodes.get_city_selection_node(),
    TripPlannerNodes.get_extract_chosen_city_node(),
    TripPlannerNodes.get_local_expert_node(),
    TripPlannerNodes.get_travel_concierge_node()
])

list_of_cities = input("Enter a list of cities (comma-separated): ").split(",")
start_date = input("Enter start date (DD Month YYYY): ")
end_date = input("Enter end date (DD Month YYYY): ")

print("\n🗓️ Planning your trip...\n")

# Full context
context = {
    "list_of_cities": list_of_cities,
    "dates": (start_date, end_date),
    "location": "Vienna",
    "interests": "art, history, and food"
}
self.logger.info("Context: %s", context)

start_time = time.perf_counter()
result = main_pipeline.run(context)
elapsed_time = time.perf_counter() - start_time

```

Figure 8: Code Snippet of main function of trip-planner/pipeline.py

- **User Input:** The pipeline collects a comma-separated list of cities and travel dates (start and end) from the user via console input.
- **Context Dictionary:** All inputs are aggregated into a context dictionary containing `list_of_cities`, `dates`, and optional parameters such as `location` and `interests`. This context is then passed to `main_pipeline.run(context)` to execute the complete workflow.

4.5 tools Directory

The `tools/` directory provides utility classes for operations used by pipeline nodes. The operations are implemented as simple functions; Figure 9 illustrates this with an example function that performs the subtraction of two numbers.

```

class Subtractor:
    @staticmethod
    def subtract(a: float, b: float) -> float:
        """Subtract two numbers and return the difference."""
        return a - b

```

Figure 9: Example of tool creation (subtraction function)

The functions in `websearch.py` (DuckDuckGo search) and `scraper.py` (HTML parsing using `BeautifulSoup`) are used as function nodes in this framework. The remaining files – `browser_tools_crewai.py` and `search_tools_crewai.py` – were adapted from the CrewAI Trip Planner tools²¹ and modified to work with the packages and local LLM configuration used in this project (DDGS and `BeautifulSoup`). This enables CrewAI agents to access the same tools, allowing a fair comparison between this framework and the standard CrewAI orchestration.

4.6 Other Files

Supporting directories include `docs/` for documentation, `scripts/` for Bash utilities used to run and test pipelines, and `logs/` for locally stored execution logs. The `docs/` folder contains Markdown files that document the project architecture, API reference, and usage examples, which can be built into a static documentation site.

The `scripts/` directory provides shell scripts for common operational tasks such as starting the `Ollama` server, running individual example pipelines, and executing test suites with appropriate environment setup.

The `logs/` folder stores timestamped execution logs generated during pipeline runs. These logs enable debugging and performance analysis by preserving the output produced during each node’s execution.

Configuration and dependency management is handled via the root-level `pyproject.toml`, which defines package metadata, Python version requirements, and dependencies, including `CrewAI`, DuckDuckGo search, and `BeautifulSoup` for web scraping.

5 How to Use

This chapter outlines the main principles for using the python PyPI-package²² deployed by the project repository. The source of this chapter is the Project README²³.

5.1 Requirements

- OS: Ubuntu 20.04 or later
- RAM: 8 GB minimum (16 GB recommended)
- Disk: 10 GB free space
- `curl` (any recent version)

²¹ *Tools of CrewAI Trip Planner*: https://github.com/crewAIInc/crewAI-examples/tree/main/crews/trip_planner/tools

²² *Project PyPI package*:: <https://pypi.org/project/pipeline-node-agents/>

²³ *Project README file*: https://github.com/ser0vs/pipeline_node_agents/blob/master/README.md

- Python 3.11–3.13

5.2 Setup Ollama

- **Install Ollama** (if not already installed):

```
curl -fsSL https://ollama.com/install.sh | sh
```

- **Run Ollama in a separate terminal:**

```
ollama serve
```

- **Install the LLM** (default: llama3.2:latest):

```
ollama pull llama3.2:latest
```

- **Verify** that the required LLM is installed:

```
ollama list
```

Expected output:

NAME	ID	SIZE	MODIFIED
llama3.2:latest	9f1c3d6a5b8e	2.0 GB	1 minute ago

5.3 Install Package

- *Optional (recommended):* create and activate a Python virtual environment:

```
python3 -m venv .venv && source .venv/bin/activate
```

- Install the package:

```
pip install pipeline-node-agents
```

5.4 Verify Installation

```
from pipeline_node_agents import greet

print(greet())
```

Expected output:

```
Hello, World! Pipeline Node Agents <version> is working.
```

5.5 Run Pipelines

5.5.1 Option 1: With Defined Model and Logger (Recommended)

```
from crewai import LLM
from pipeline_node_agents import init_pipeline_logger, get_logger, TripPlannerPipeline

init_pipeline_logger(pipeline_name="trip_planner_pipeline", project_root=".")
logger = get_logger(__name__)

# Default model is llama3.2, change if needed
ollama_llm = LLM(model="ollama/llama3.2", base_url="http://localhost:11434")

pipeline = TripPlannerPipeline(ollama_llm=ollama_llm, logger=logger)
pipeline.run()
```

5.5.2 Option 2: Without Logger

```
from pipeline_node_agents import TripPlannerPipeline

# Default model is llama3.2
pipeline = TripPlannerPipeline()
pipeline.run()
```

5.5.3 Available tools configuration

By default, agent nodes work as pure LLMs (without tools). You can allow them to search for information by themselves.

Note: Usage of tools is not recommended for light-weight LLMs (< 8 billion hyperparameters).

To enable tools for specific agents in `TripPlannerPipeline`:

1. Enable tools for the local expert agent only:

```
pipeline = TripPlannerPipeline(local_expert_tools_enabled=True)
```

2. Enable tools for the travel concierge agent only:

```
pipeline = TripPlannerPipeline(travel_concierge_tools_enabled=True)
```

3. Enable tools for both agents:

```
pipeline = TripPlannerPipeline(
    local_expert_tools_enabled=True,
    travel_concierge_tools_enabled=True
)
```

6 How to Maintain

This chapter presents the key guidelines for working with the project repository. The content of this chapter is based on the Project maintenance_guide²⁴.

6.1 Creating a Function Node

```
from pipeline_node_agents.adapters.python_fn_adapter import PythonFnAdapter
from pipeline_node_agents.core.node import FunctionNode

def my_function(input_value: int) -> int:
    return input_value * 2

node = FunctionNode(
    name="MyNode",
    adapter=PythonFnAdapter(my_function),
    inputs=["input_value"],
    output="output_value"
)
```

6.2 Creating an Agent Node

```
from pipeline_node_agents.core.node import AgentNode

node = AgentNode(
    name="MyAgentNode",
    adapter=your_adapter, # Use an appropriate adapter from adapters/
    inputs=["data"],
    output="result"
)
```

6.3 Building a Pipeline

Based on the project structure, newly created pipelines should be placed in the `src/pipeline_node_agents/examples` directory.

Option 1: Linear Pipeline (list of nodes) For simple sequential pipelines, a list of nodes can be passed directly to the pipeline constructor:

```
from pipeline_node_agents.core.pipeline import Pipeline

pipeline = Pipeline(nodes=[node1, node2, node3])
result = pipeline.run(initial_context={"input_value": 5})
```

This automatically creates sequential edges: `node1 -> node2 -> node3`.

²⁴Project maintenance guide:: https://github.com/ser0vs/pipeline_node_agents/blob/master/docs/maintenance_guide.md

Option 2: Graph-based Pipeline (start node + edges) For pipelines requiring conditional branching or more complex execution flows:

```
from pipeline_node_agents.core.pipeline import Pipeline

# Create pipeline with a start node
pipeline = Pipeline(start_node=decision_node)

# Add additional nodes
pipeline.add_node(branch_a_node)
pipeline.add_node(branch_b_node)

# Add conditional edges
pipeline.add_edge(
    "DecisionNode",
    "BranchANode",
    condition=lambda ctx: ctx["choice"] == "A"
)
pipeline.add_edge(
    "DecisionNode",
    "BranchBNode",
    condition=lambda ctx: ctx["choice"] == "B"
)

result = pipeline.run(initial_context={"choice": "A"})
```

6.3.1 Merging Pipelines

An existing pipeline can be merged into another pipeline instance:

```
sub_pipeline = Pipeline(nodes=[search_node, scrape_node, summarize_node])

main_pipeline = Pipeline(start_node=start_node)
main_pipeline.add_pipeline(sub_pipeline)
main_pipeline.add_edge("StartNode", "SearchNode")
```

6.3.2 Nested Pipelines

Pipelines can be executed inside a function node to implement complex sub-workflows:

```
def process_items(items: list[str]) -> dict:
    """Run a sub-pipeline for each item and aggregate results."""
    results = {}

    for item in items:
        # Define nodes for sub-pipeline
```

```

        # search_node = FunctionNode(...)
        # process_node = AgentNode(...)

        sub_pipeline = Pipeline(nodes=[search_node, process_node])
        result = sub_pipeline.run({"query": item})
        results[item] = result.get("output")

    return {"aggregated_results": results}

# Use the function as a node in the main pipeline
nested_node = FunctionNode(
    name="NestedPipelineNode",
    adapter=PythonFnAdapter(process_items),
    inputs=["items"],
    output="aggregated_results"
)

main_pipeline = Pipeline(nodes=[nested_node, final_node])

```

This pattern is useful when pipelines must be created dynamically based on runtime data.

6.3.3 Edge Conditions

- **Unconditional edge:** `condition=None` (default), meaning the edge is always followed.
- **Conditional edge:** `condition=lambda ctx: <bool>`, meaning the edge is followed only if the condition evaluates to `True`.

When multiple outgoing edges exist, the first edge whose condition evaluates to `True` is selected.

6.3.4 Creating Loops

Loops can be created by adding edges that point back to previously executed nodes:

```

pipeline.add_edge(
    "ProcessNode",
    "CheckNode",
    condition=lambda ctx: ctx["retry_count"] < 3
)
pipeline.add_edge(
    "CheckNode",
    "ProcessNode",
    condition=lambda ctx: not ctx["success"]
)

```

```

)
pipeline.add_edge(
    "CheckNode",
    "FinalNode",
    condition=lambda ctx: ctx["success"]
)

```

6.3.5 Creating Adapters

Custom agent frameworks can be integrated by implementing a custom adapter. To do so, inherit from `BaseAdapter` and implement the `invoke` method.

The `invoke` method may accept arbitrary input fields, but it must return exactly one output value.

6.4 Run Test Scripts

During development, it is essential to ensure that new changes do not impact existing functionality. For this reason, bash-based test scripts were introduced.

Prerequisites: Completed Ollama setup as described in Section 5.2 – Setup Ollama.

- **Install Poetry** (if not already installed):

```
curl -sSL https://install.python-poetry.org | python3 -
```

- **Clone the repository and install dependencies:**

```

git clone <repository_url>
cd pipeline_node_agents
poetry install

```

6.4.1 Option 1: Run a Single Pipeline

```
./scripts/run_single_pipeline.sh <path_to_pipeline> <runs> [input_strings]
```

Parameters

- `<path_to_pipeline>` – Path to the Python pipeline file
- `<runs>` – Number of times the pipeline is executed
- `[input_strings]` – Optional: newline-separated inputs for interactive prompts

6.4.2 Option 2: Run All Example Pipelines (Smoke Tests)

```
./scripts/run_smoke_pipelines.sh [number_of_runs_per_pipeline]
```


7 Evaluation

The primary goal of the evaluation was to compare the Trip Planner pipeline implemented using the Pipeline Node Agents framework against the existing Trip Planner example from CrewAI, testing both implementations with at least two different local LLM models with the same tools available for agents.

7.1 Evaluation Approach

For this evaluation, Google Colab²⁵ was used as the execution environment, with two separate notebooks created: one for CrewAI experiments and one for Pipeline Node Agents experiments. The models tested were Qwen3:8b and Llama3.2:latest (3b), both served locally via Ollama.

Google Colab was selected as the execution environment after considering several alternatives such as Google Cloud VM, Amazon EC2, and Azure Virtual Machines; however, none of these platforms provide free GPU access even for short-duration experiments, whereas Google Colab offers free T4 GPU runtime sufficient for running 8B parameter models. Additionally, Google Colab’s accessibility is advantageous since virtually everyone has a Google account, eliminating the need for additional account setup or credit card registration.

To run the experiments, users must upload the notebooks to Google Drive, open them in Google Colab with a T4 GPU runtime (or higher), and execute the dependency installation cells before following the instructions in section 6 of each notebook; detailed requirements and troubleshooting steps are available in the experiments README²⁶. To switch between models in the CrewAI experiment, run the command (replacing with the desired model name) before execution:

```
export OLLAMA_MODEL="qwen3:8b"
```

For the Pipeline Node Agents experiment, modify line 10 in the configuration file:

```
src/pipeline_node_agents/examples/trip_planner/config.py:10
```

to specify the desired model.

7.2 Preparation

For CrewAI compatibility with local LLMs, a fork of the official CrewAI examples repository was created²⁷, with modifications made in the branch `trip_planner_all_tools`²⁸

²⁵ Google Colab: <https://colab.google/>

²⁶ README of project experiments: https://github.com/ser0vs/pipeline_node_agents/blob/master/experiments/README.md

²⁷ Fork of repository of CrewAI created for evaluation: <https://github.com/ser0vs/crewAI-examples>

²⁸ Branch for experiments of CrewAI trip planner with all tools enabled: https://github.com/ser0vs/crewAI-examples/tree/trip_planner_all_tools

to enable local model support. Additional branches were introduced to control the number of tools available to agents; these will be discussed later.

The CrewAI Trip Planner offers three agent tools: web search, scraping, and a calculator. To ensure a fair comparison, the web search and scraping tools were adapted to support local LLM execution and the libraries used in this project (Pipeline Node Agents, hereafter referred to as PNA). DuckDuckGo was used in place of Serper, which requires a restricted API, and BeautifulSoup was employed for web scraping. The calculator tool was transferred to PNA without modification.

To set up tools of agents in PNA, parameters `local_expert_tools_enabled` and `travel_concierge_tools_enabled` can be set in the constructor of `TripPlannerPipeline` (according to readme README).

To adapt tools of agents in CrewAI trip planner, several branches were created: `trip_planner_local_expert_tools_only`, `trip_planner_all_tools` and `trip_planner_no_tools`.

7.3 Evaluation Results

All logs of evaluation are available in the folder `/logs/supplemental_materials`²⁹. In all experiments, the city list always includes exactly two options – Madrid and Dubai – and the pipeline must select one of them for the trip itinerary: the "Pipeline Node Agents" (PNA) based on weather conditions and CrewAI based on weather and ticket prices. In all experiments, date range is provided from 14th to 16th of February 2026.

7.3.1 Qwen3 Model with All Tools

In this experiment, tools are enabled for all agents except the City Selector Agent in PNA, since, according to the pipeline architecture, it makes its decision solely based on the information supplied by the preceding function node. As provided in Table 1: PNA completes in average 249 seconds while CrewAI averages 434 seconds, which is almost twice slower than PNA. That may be partly due to simplified instructions of PNA, but the bigger time consumer is city selection. In CrewAI, agent decides his own which factors to consider and which parameters to search separately. PNA takes decision based on provided information which is always up-to-date.

A difference in output quality is observable but not critical: CrewAI consistently generates more detailed responses, including itemized budget tables, whereas PNA omits explicit expense breakdowns. This variation results from differences in prompt complexity — PNA’s prompts are optimized for Llama 3.2’s concise style and therefore provide insufficient guidance for Qwen 3, which requires more explicit instructions.

²⁹ Logs of experiments (CrewAI and "Pipeline Node Agents"): https://github.com/ser0vs/pipeline_node_agents/tree/master/logs/supplemental_materials

Run	Pipeline Node Agents	CrewAI
1	223.68	288.94
2	228.55	342.98
3	224.53	423.41
4	236.46	383.93
5	320.85	481.49
6	260.78	590.10
7	291.48	381.13
8	219.92	588.08
9	266.18	317.78
10	214.93	541.61
Mean	248.74	433.95

Table 1: Execution Times for Trip Planner Pipelines on Qwen3:8b (seconds) with all tools available for agents

7.3.2 Llama3.2 Model with All Tools

In this scenario, both the CrewAI and PNA trip planners were executed with full access to all available tools.

In both cases, the systems failed to produce meaningful results, as agents became trapped in repetitive loops—repeatedly issuing identical search queries and generating incoherent responses, such as “Compare the estimated monthly costs for both cities,” instead of actual travel recommendations. Execution times increased significantly and often required manual termination due to the models’ inability to reason effectively about tool usage.

These results demonstrate that, without restricting tool access, agents are unable to produce reasonable outputs. Consequently, subsequent experiments selectively remove tools from certain agents.

7.3.3 Llama3.2 Model without Tools

Let us disable all tools and treat the agents as plain LLMs. In this configuration, PNA has an advantage because its Function Nodes still provide up-to-date information. Despite the relatively long city-research process in PNA, it executes almost 30% faster than CrewAI: as shown in Table 2, after 10 runs PNA achieves an average execution time of 43.63 seconds, compared to 58.91 seconds for CrewAI. This difference can be attributed not only to prompt complexity, but also to the orchestration overhead introduced by CrewAI, which lightweight local models experience more strongly than large cloud-based models. PNA, in contrast, forwards only the context strictly required by the subsequent agent and therefore achieves higher execution speed.

Run	Pipeline Node Agents	CrewAI
1	38.37	52.81
2	43.98	63.12
3	41.57	50.71
4	40.85	48.79
5	48.58	61.17
6	44.06	73.76
7	47.85	70.66
8	48.42	46.81
9	37.21	60.26
10	45.36	60.98
Mean	43.63	58.91

Table 2: Execution Times for Trip Planner Pipelines on Llama3.2:latest without tools available for agents (seconds)

7.3.4 Llama3.2 Model with Tools for Local Expert only

After verifying that the pipelines operate correctly without tools, tools were enabled for only a single agent —the Local Expert, which occupies the second position in the CrewAI pipeline. In this setup, the final agent (Travel Concierge) terminates the chain and produces the final response, even if the preceding Local Expert becomes confused by tool usage and fails to generate a meaningful output. Across 10 runs, the tool-enabled agent exhibited instability in approximately three cases on average, returning internal reasoning, search queries, or similar intermediate artifacts instead of usable content. Nevertheless, because the final agent operates without tools, the overall output remains coherent. This experiment demonstrates that even limited tool usage can be unstable when using lightweight models. A potential mitigation is the introduction of additional Function Nodes (in the case of PNA) to provide context in a deterministic manner.

8 Conclusion and Future Work

In conclusion, PNA achieves performance comparable to CrewAI across most experiments and, in certain cases, surpasses it with respect to execution speed and suitability for lightweight LLMs. Its graph-based orchestration is especially advantageous in settings where agent access to tools is constrained. The inclusion of function nodes enables deterministic processing, thereby limiting agents’ dependence on tools. In contrast, CrewAI’s more heavyweight orchestration introduces additional overhead that remains noticeable even when relatively powerful models such as Qwen3:8b are used.

However, PNA is more challenging to maintain than CrewAI. CrewAI offers a more general and easily adaptable framework for diverse use cases. This distinction arises from architectural differences: PNA’s graph-based orchestration

demands a more precisely defined workflow, which increases maintenance effort but also allows for finer-grained control and tuning.

8.1 Limitations

The primary limitation of this framework is that it performs best when tasks are closely related and depend on similar external context; if tasks vary significantly, additional tools must be provided for Agent nodes, potentially reducing the framework's efficiency advantage over more flexible agentic approaches. The second limitation is that the framework currently only handles text-based inputs and outputs, and does not yet offer multimodal support for images, audio, or other data types. Furthermore, testing is still difficult because both external tools (for example, web search results that change over time) and LLM outputs are inherently non-deterministic, which complicates the creation of stable regression tests and reproducible benchmarks.

8.2 Future Work

Since the project already defines the core framework, its possible extensions are essentially limitless:

- adapters for any type of LLM or agent framework can be implemented by subclassing the BaseAgent class
- multimodal capabilities can be added to handle audio, video, and other data formats
- more powerful LLMs can be integrated, including external ones accessed via API
- eventually, a user-friendly UI for the framework can be developed to visualize and trace workflows.

By releasing it as open source, I'm excited to discover what kinds of projects people will build on top of this framework.

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [2] Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, Junxiang Wang, Tanmoy Chowdhury, Yun Li, Hejie Cui, Xuchao Zhang, et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *ACM Computing Surveys*, 58(3):1–39, 2025.
- [3] Aaron Xuxiang Tian, Ruofan Zhang, Jiayao Tang, Y. Cho, Xueqian Li, Qiang Yi, Ji Wang, Zhunping Zhang, Danrui Qi, S. Guntuku, Lyle Ungar, Tianyu Shi, and Chi Wang. Beyond the strongest llm: Multi-turn multi-agent orchestration vs. single llms on benchmarks. *ArXiv*, abs/2509.23537, 2025.
- [4] Chaojia Yu, Zihan Cheng, Hanwen Cui, Yishuo Gao, Zexu Luo, Yijin Wang, Hangbin Zheng, and Yong Zhao. A survey on agent workflow—status and future. In *2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD)*, pages 770–781. IEEE, 2025.
- [5] Robert St Amant and Alexander B Wood. Tool use for autonomous agents. In *AAAI*, pages 184–189, 2005.
- [6] Satyadhar Joshi. Review of autonomous systems and collaborative ai agent frameworks. *International Journal of Science and Research Archive*, 2025.
- [7] Hana Derouiche, Zaki Brahmi, and Haithem Mazeni. Agentic ai frameworks: Architectures, protocols, and design challenges. *ArXiv*, abs/2508.10146, 2025.